01 LC 20 | Valid Parentheses (Easy)

Problem: Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid. An input string is valid if: - Open brackets must be closed by the same type of brackets. - Open brackets must be closed in the correct order. - Every close bracket has a corresponding open bracket of the same type.

Constraints: 1 <= s.length <= 10^4; s consists of parentheses only '()[]{}'.

```
class Solution {
public:
  bool isValid(string s) {
    string stk;
    unordered_map<char, char> d{{'(', ')'}, {'[',
      ']'}, {'{', '}'}};
    for (char c : s) {
      if (d.contains(c)) {
        stk.push_back(d[c]);
      } else if (stk.empty() || stk.back() != c) {
        return false;
      } else {
        stk.pop_back();
      }
    }
    return stk.empty();
  }
};
```

43 LC 994 | Rotting Oranges (Medium)

Problem: You are given an m x n grid where each cell can have one of three values: - 0 representing an empty cell, - 1 representing a fresh orange, or - 2 representing a rotten orange. Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten. Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Constraints: m == grid.length; n == grid[i].length; 1 <= m, n <= 10; grid[i][j] is 0, 1, or 2.

```
class Solution {
public:
  int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int, int>> q;
    int cnt = 0;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (grid[i][j] == 1) {
          ++cnt;
        } else if (grid[i][j] == 2) {
          q.emplace(i, j);
        }
      }
    }
    const int dirs[5] = {-1, 0, 1, 0, -1};
    for (int ans = 1; q.size() && cnt; ++ans) {
      for (int k = q.size(); k; --k) {
        auto [i, j] = q.front();
        q.pop();
        for (int d = 0; d < 4; ++d) {
          int x = i + dirs[d], y = j + dirs[d +
            1];
          if (x >= 0 && x < m && y >= 0 && y < n
            && grid[x][y] == 1) {
            grid[x][y] = 2;
            q.emplace(x, y);
            if (--cnt == 0) {
              return ans;
            }
          }
        }
      }
    }
    return cnt > 0 ? -1 : 0;
  }
};
```

02 LC 155 | Min Stack (Medium)

Problem: Implement MinStack with push(val), pop(), top(), and getMin(). Return the current top/minimum as requested. Every operation must take O(1) time; pop, top, and getMin are called only on nonempty stacks.

Constraints: -2^31 <= val <= 2^31 - 1; Methods pop, top and getMin operations will always be called on non-empty stacks.; At most 3 * 10^4 calls will be made to push, pop, top, and getMin.

```
class MinStack {
public:
  MinStack() {
    stk2.push(INT_MAX);
  }
  void push(int val) {
    stk1.push(val);
    stk2.push(min(val, stk2.top()));
  }
  void pop() {
    stk1.pop();
    stk2.pop();
  }
  int top() {
    return stk1.top();
  }
  int getMin() {
    return stk2.top();
  }
private:
  stack<int> stk1;
  stack<int> stk2;
};
```

20 LC 853 | Car Fleet (Medium)

Problem: Cars at distinct positions drive toward target without passing. A faster following car joins the slower car/fleet ahead, including when they meet exactly at target. Return the number of fleets arriving.

Constraints: n == position.length == speed.length; 1 <= n <= 10^5; 0 < target <= 10^6; 0 <= position[i] < target; All the values of position are unique.; 0 < speed[i] <= 10^6

```
class Solution {
public:
  int carFleet(int target, vector<int>& position,
    vector<int>& speed) {
    int n = position.size();
    vector<int> idx(n);
    iota(idx.begin(), idx.end(), 0);
    ranges::sort(idx, [&](int i, int j) {
      return position[i] > position[j];
    });
    int ans = 0;
    double pre = 0;
    for (int i : idx) {
      double t = 1.0 * (target - position[i]) /
        speed[i];
      if (t > pre) {
        ++ans;
        pre = t;
      }
    }
    return ans;
  }
};
```

03 LC 232 | Implement Queue using Stacks (Easy)

Problem: Implement MyQueue with push(x), pop(), peek(), and empty() using only standard stack operations. pop/peek calls are valid and must provide FIFO behavior.

Constraints: 1 <= x <= 9; At most 100 calls will be made to push, pop, peek, and empty.; All the calls to pop and peek are valid.

```
class MyQueue {
public:
  MyQueue() {
  }
  void push(int x) {
    stk1.push(x);
  }
  int pop() {
    move();
    int ans = stk2.top();
    stk2.pop();
    return ans;
  }
  int peek() {
    move();
    return stk2.top();
  }
  bool empty() {
    return stk1.empty() && stk2.empty();
  }
private:
  stack<int> stk1;
  stack<int> stk2;
  void move() {
    if (stk2.empty()) {
      while (!stk1.empty()) {
        stk2.push(stk1.top());
        stk1.pop();
      }
    }
  }
};
```

12 LC 394 | Decode String (Medium)

Problem: Decode nested expressions k[text], which repeat text exactly k times. Repeat counts may have multiple digits. The encoding is valid, contains no extra spaces, and literal text contains no digits. Decoded length is at most 10^5.

Constraints: 1 <= s.length <= 30; s consists of lowercase English letters, digits, and square brackets '[]'.; s is guaranteed to be a valid input.; All the integers in s are in the range [1, 300].

```
class Solution {
public:
  string decodeString(string s) {
    vector<pair<string, int>> stack;
    string current;
    int count = 0;
    for (unsigned char c : s) {
      if (isdigit(c)) count = count * 10 + c -
        '0';
      else if (c == '[') {
        stack.emplace_back(move(current), count);
        current.clear();
        count = 0;
      } else if (c == ']') {
        auto [prefix, repeat] =
          move(stack.back());
        stack.pop_back();
        prefix.reserve(prefix.size() +
          current.size() * repeat);
        while (repeat--) prefix += current;
        current = move(prefix);
      } else current += static_cast<char>(c);
    }
    return current;
  }
};
```

04 LC 225 | Implement Stack using Queues (Easy)

Problem: Implement MyStack with push(x), pop(), top(), and empty() using only standard FIFO queue operations. pop/top calls are valid and must provide LIFO behavior.

Constraints: 1 <= x <= 9; At most 100 calls will be made to push, pop, top, and empty.; All the calls to pop and top are valid.

```
class MyStack {
public:
  MyStack() {
  }
  void push(int x) {
    q2.push(x);
    while (!q1.empty()) {
      q2.push(q1.front());
      q1.pop();
    }
    swap(q1, q2);
  }
  int pop() {
    int x = q1.front();
    q1.pop();
    return x;
  }
  int top() {
    return q1.front();
  }
  bool empty() {
    return q1.empty();
  }
private:
  queue<int> q1;
  queue<int> q2;
};
```

46 LC 542 | 01 Matrix (Medium)

Problem: Given an m x n binary matrix mat, return the distance of the nearest 0 for each cell. The distance between two cells sharing a common edge is 1.

Constraints: m == mat.length; n == mat[i].length; 1 <= m, n <= 10^4; 1 <= m * n <= 10^4; mat[i][j] is either 0 or 1.; There is at least one 0 in mat.

```
class Solution {
public:
  vector<vector<int>>
    updateMatrix(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    vector<vector<int>> ans(m, vector<int>(n,
      -1));
    queue<pair<int, int>> q;
    for (int i = 0; i < m; ++i) {
      for (int j = 0; j < n; ++j) {
        if (mat[i][j] == 0) {
          ans[i][j] = 0;
          q.emplace(i, j);
        }
      }
    }
    vector<int> dirs = {-1, 0, 1, 0, -1};
    while (!q.empty()) {
      auto p = q.front();
      q.pop();
      for (int i = 0; i < 4; ++i) {
        int x = p.first + dirs[i];
        int y = p.second + dirs[i + 1];
        if (x >= 0 && x < m && y >= 0 && y < n &&
          ans[x][y] == -1) {
          ans[x][y] = ans[p.first][p.second] + 1;
          q.emplace(x, y);
        }
      }
    }
    return ans;
  }
};
```

05 LC 150 | Evaluate Reverse Polish Notation (Medium)

Problem: Evaluate the given valid reverse-Polish expression. Operators are +, -, *, /. Division truncates toward zero; division by zero does not occur. Intermediate values and the result fit signed 32-bit integers.
Constraints: 1 <= tokens.length <= 10^4; tokens[i] is either an operator: "+", "-", "*", or "/", or an integer in the range [-200, 200].

```
class Solution {
public:
  int evalRPN(vector<string>& tokens) {
    stack<int> stk;
    for (auto& t : tokens) {
      if (t.size() > 1 || isdigit(t[0])) {
        stk.push(stoi(t));
      } else {
        int y = stk.top();
        stk.pop();
        int x = stk.top();
        stk.pop();
        if (t[0] == '+')
          stk.push(x + y);
        else if (t[0] == '-')
          stk.push(x - y);
        else if (t[0] == '*')
          stk.push(x * y);
        else
          stk.push(x / y);
      }
    }
    return stk.top();
  }
};
```

35 LC 1438 | Longest Continuous Subarray With Absolute Diff Less Than or Equal to Limit (Medium)

Problem: You are given an array of integers nums and an integer limit. Return the size of the longest non-empty subarray such that the absolute difference between every pair of elements in this subarray is less than or equal to limit.
Constraints: 1 <= nums.length <= 10^5; 1 <= nums[i] <= 10^9; 0 <= limit <= 10^9

```
class Solution {
public:
  int longestSubarray(vector<int>& nums, int
    limit) {
    deque<int> max_q;
    deque<int> min_q;
    int n = nums.size();
    int l = 0;
    for (int r = 0; r < n; ++r) {
      while (!max_q.empty() && nums[max_q.back()]
        < nums[r]) {
        max_q.pop_back();
      }
      while (!min_q.empty() && nums[min_q.back()]
        > nums[r]) {
        min_q.pop_back();
      }
      max_q.push_back(r);
      min_q.push_back(r);
      if (nums[max_q.front()] -
        nums[min_q.front()] > limit) {
        ++l;
        if (max_q.front() < l) {
          max_q.pop_front();
        }
        if (min_q.front() < l) {
          min_q.pop_front();
        }
      }
    }
    return n - l;
  }
};
```

06 LC 739 | Daily Temperatures (Medium)

Problem: Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.
Constraints: 1 <= temperatures.length <= 10^5; 30 <= temperatures[i] <= 100

```
class Solution {
public:
  vector<int> dailyTemperatures(vector<int>&
    temperatures) {
    int n = temperatures.size();
    stack<int> stk;
    vector<int> ans(n);
    for (int i = n - 1; ~i; --i) {
      while (!stk.empty() &&
        temperatures[stk.top()] <=
        temperatures[i]) {
        stk.pop();
      }
      if (!stk.empty()) {
        ans[i] = stk.top() - i;
      }
      stk.push(i);
    }
    return ans;
  }
};
```

10 LC 85 | Maximal Rectangle (Hard)

Problem: Given a rows x cols binary matrix filled with 0's and 1's, find the largest rectangle containing only 1's and return its area.
Constraints: rows == matrix.length; cols == matrix[i].length; 1 <= rows, cols <= 200; matrix[i][j] is '0' or '1'.

```
class Solution {
public:
  int maximalRectangle(vector<vector<char>>&
    matrix) {
    int n = matrix[0].size();
    vector<int> heights(n);
    int ans = 0;
    for (auto& row : matrix) {
      for (int j = 0; j < n; ++j) {
        if (row[j] == '1')
          ++heights[j];
        else
          heights[j] = 0;
      }
      ans = max(ans,
        largestRectangleArea(heights));
    }
    return ans;
  }
  int largestRectangleArea(vector<int>& heights) {
    int res = 0, n = heights.size();
    stack<int> stk;
    vector<int> left(n, -1);
    vector<int> right(n, n);
    for (int i = 0; i < n; ++i) {
      while (!stk.empty() && heights[stk.top()] >=
        heights[i]) {
        right[stk.top()] = i;
        stk.pop();
      }
      if (!stk.empty()) left[i] = stk.top();
      stk.push(i);
    }
    for (int i = 0; i < n; ++i)
      res = max(res, heights[i] * (right[i] -
        left[i] - 1));
    return res;
  }
};
```

07 LC 496 | Next Greater Element I (Easy)

Problem: For each value in distinct nums1, find its matching position in distinct nums2 and return the first strictly greater value to its right, or -1. nums1 is a subset of nums2.

Constraints: 1 <= nums1.length <= nums2.length <= 1000; 0 <= nums1[i], nums2[i] <= 10^4; All integers in nums1 and nums2 are unique.; All the integers of nums1 also appear in nums2.

```
class Solution {
public:
  vector<int> nextGreaterElement(vector<int>&
    nums1, vector<int>& nums2) {
    stack<int> stk;
    unordered_map<int, int> d;
    ranges::reverse(nums2);
    for (int x : nums2) {
      while (!stk.empty() && stk.top() < x) {
        stk.pop();
      }
      if (!stk.empty()) {
        d[x] = stk.top();
      }
      stk.push(x);
    }
    vector<int> ans;
    for (int x : nums1) {
      ans.push_back(d.contains(x) ? d[x] : -1);
    }
    return ans;
  }
};
```

45 LC 199 | Binary Tree Right Side View (Medium)

Problem: Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Constraints: The number of nodes in the tree is in the range [0, 100].; -100 <= Node.val <= 100

```
class Solution {
public:
  vector<int> rightSideView(TreeNode* root) {
    vector<int> ans;
    if (!root) {
      return ans;
    }
    queue<TreeNode*> q{{root}};
    while (q.size()) {
      ans.push_back(q.front()->val);
      for (int k = q.size(); k; --k) {
        auto node = q.front();
        q.pop();
        if (node->right) {
          q.push(node->right);
        }
        if (node->left) {
          q.push(node->left);
        }
      }
    }
    return ans;
  }
};
```

08 LC 503 | Next Greater Element II (Medium)

Problem: Given a circular integer array nums (i.e., the next element of nums[nums.length - 1] is nums[0]), return the next greater number for every element in nums. The next greater number of a number x is the first greater number to its traversing-order next in the array, which means you could search circularly to find its next greater number. If it doesn't exist, return -1 for this number.

Constraints: 1 <= nums.length <= 10^4; -10^9 <= nums[i] <= 10^9

```
class Solution {
public:
  vector<int> nextGreaterElements(vector<int>&
    nums) {
    int n = nums.size();
    vector<int> ans(n, -1);
    stack<int> stk;
    for (int i = n * 2 - 1; ~i; --i) {
      int j = i % n;
      while (stk.size() && stk.top() <= nums[j]) {
        stk.pop();
      }
      if (stk.size()) {
        ans[j] = stk.top();
      }
      stk.push(nums[j]);
    }
    return ans;
  }
};
```

17 LC 402 | Remove K Digits (Medium)

Problem: Given string num representing a non-negative integer num, and an integer k, return the smallest possible integer after removing k digits from num.

Constraints: 1 <= k <= num.length <= 10^5; num consists of only digits.; num does not have any leading zeros except for the zero itself.

```
class Solution {
public:
  string removeKdigits(string num, int k) {
    string stk;
    for (char& c : num) {
      while (k && stk.size() && stk.back() > c) {
        stk.pop_back();
        --k;
      }
      stk += c;
    }
    while (k--) {
      stk.pop_back();
    }
    int i = 0;
    for (; i < stk.size() && stk[i] == '0'; ++i) {
    }
    string ans = stk.substr(i);
    return ans == "" ? "0" : ans;
  }
};
```

09 LC 84 | Largest Rectangle in Histogram (Hard)
Problem: Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.
Constraints: 1 <= heights.length <= 10^5; 0 <= heights[i] <= 10^4

```
class Solution {
public:
  int largestRectangleArea(vector<int>& heights) {
    int res = 0, n = heights.size();
    stack<int> stk;
    vector<int> left(n, -1);
    vector<int> right(n, n);
    for (int i = 0; i < n; ++i) {
      while (!stk.empty() && heights[stk.top()] >=
        heights[i]) {
        right[stk.top()] = i;
        stk.pop();
      }
      if (!stk.empty()) left[i] = stk.top();
      stk.push(i);
    }
    for (int i = 0; i < n; ++i)
      res = max(res, heights[i] * (right[i] -
        left[i] - 1));
    return res;
  }
};
```

31 LC 933 | Number of Recent Calls (Easy)
Problem: Implement RecentCounter.ping(t). Record a request at strictly increasing time t and return how many requests occurred in the inclusive interval [t-3000,t].
Constraints: 1 <= t <= 10^9; Each test case will call ping with strictly increasing values of t.; At most 10^4 calls will be made to ping.

```
class RecentCounter {
public:
  queue<int> q;
  RecentCounter() {
  }
  int ping(int t) {
    q.push(t);
    while (q.front() < t - 3000) q.pop();
    return q.size();
  }
};
```

38 LC 2073 | Time Needed to Buy Tickets (Easy)
Problem: People queue to buy tickets. Each second the front person buys one ticket, leaving if finished or rejoining the back otherwise. Return the seconds until the person originally at index k finishes all requested tickets.
Constraints: n == tickets.length; 1 <= n <= 100; 1 <= tickets[i] <= 100; 0 <= k < n

```
class Solution {
public:
  int timeRequiredToBuy(vector<int>& tickets, int
    k) {
    int ans = 0;
    for (int i = 0; i < tickets.size(); ++i) {
      ans += min(tickets[i], i <= k ? tickets[k] :
        tickets[k] - 1);
    }
    return ans;
  }
};
```

11 LC 42 | Trapping Rain Water (Hard)
Problem: Bars have heights height[i] and unit width. Return the total water trapped between the bars after rain.
Constraints: n == height.length; 1 <= n <= 2 * 10^4; 0 <= height[i] <= 10^5

```
class Solution {
public:
  int trap(vector<int>& height) {
    int left = 0, right =
      static_cast<int>(height.size()) - 1;
    int leftMax = 0, rightMax = 0, water = 0;
    while (left <= right) {
      if (height[left] <= height[right]) {
        leftMax = max(leftMax, height[left]);
        water += leftMax - height[left++];
      } else {
        rightMax = max(rightMax, height[right]);
        water += rightMax - height[right--];
      }
    }
    return water;
  }
};
```

19 LC 901 | Online Stock Span (Medium)
Problem: Implement StockSpanner.next(price), returning today's stock span: the number of consecutive days ending today whose prices are at most today's price, including today itself.
Constraints: 1 <= price <= 10^5; At most 10^4 calls will be made to next.

```
class StockSpanner {
public:
  StockSpanner() {
  }
  int next(int price) {
    int cnt = 1;
    while (!stk.empty() && stk.top().first <=
      price) {
      cnt += stk.top().second;
      stk.pop();
    }
    stk.emplace(price, cnt);
    return cnt;
  }
private:
  stack<pair<int, int>> stk;
};
```

50 LC 2390 | Removing Stars From a String (Medium)
Problem: You are given a string s, which contains stars *. In one operation, you can: - Choose a star in s. - Remove the closest non-star character to its left, as well as remove the star itself. Return the string after all stars have been removed. Note: - The input will be generated such that the operation is always possible. - It can be shown that the resulting string will always be unique.
Constraints: 1 <= s.length <= 10^5; s consists of lowercase English letters and stars *.; The operation above can be performed on s.

```
class Solution {
public:
  string removeStars(string s) {
    string ans;
    for (char c : s) {
      if (c == '*') {
        ans.pop_back();
      } else {
        ans.push_back(c);
      }
    }
    return ans;
  }
};
```

13 LC 71 | Simplify Path (Medium)

Problem: Simplify an absolute Unix path: collapse repeated slashes, ignore '.', resolve '..' to the parent without moving above root, and remove trailing slashes except for '/'. Other names, including '...', are ordinary directory names.

Constraints: 1 <= path.length <= 3000; path consists of English letters, digits, period '.', slash '/' or '_'.; path is a valid absolute Unix path.

```
class Solution {
public:
  string simplifyPath(string path) {
    deque<string> stk;
    stringstream ss(path);
    string t;
    while (getline(ss, t, '/')) {
      if (t == "" || t == ".") {
        continue;
      }
      if (t == "..") {
        if (!stk.empty()) {
          stk.pop_back();
        }
      } else {
        stk.push_back(t);
      }
    }
    if (stk.empty()) {
      return "/";
    }
    string ans;
    for (auto& s : stk) {
      ans += "/" + s;
    }
    return ans;
  }
};
```

25 LC 1249 l Minimum Remove to Make Valid Parentheses (Medium)

Problem: Remove the fewest parentheses from s so its parentheses are balanced; keep all other characters and preserve order. Return any valid result.

Constraints: 1 <= s.length <= 10^5; s[i] is either '(' , ')', or lowercase English letter.

```
class Solution {
public:
  string minRemoveToMakeValid(string s) {
    string stk;
    int x = 0;
    for (char& c : s) {
      if (c == ')' && x == 0) continue;
      if (c == '(')
        ++x;
      else if (c == ')')
        --x;
      stk.push_back(c);
    }
    string ans;
    x = 0;
    while (stk.size()) {
      char c = stk.back();
      stk.pop_back();
      if (c == '(' && x == 0) continue;
      if (c == ')')
        ++x;
      else if (c == '(')
        --x;
      ans.push_back(c);
    }
    ranges::reverse(ans);
    return ans;
  }
};
```

14 LC 227 | Basic Calculator II (Medium)

Problem: Given a string s which represents an expression, evaluate this expression and return its value. The integer division should truncate toward zero. You may assume that the given expression is always valid. All intermediate results will be in the range of [-2^31, 2^31 - 1]. Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Constraints: 1 <= s.length <= 3 * 10^5; s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.; s represents a valid expression.; All the integers in the expression are non-negative integers in the range [0, 2^31 - 1].; The answer is guaranteed to fit in a 32-bit integer.

```
class Solution {
public:
  int calculate(string s) {
    int v = 0, n = s.size();
    char sign = '+';
    stack<int> stk;
    for (int i = 0; i < n; ++i) {
      char c = s[i];
      if (isdigit(c)) v = v * 10 + (c - '0');
      if (i == n - 1 || c == '+' || c == '-' || c
        == '*' || c == '/') {
        if (sign == '+')
          stk.push(v);
        else if (sign == '-')
          stk.push(-v);
        else if (sign == '*') {
          int t = stk.top();
          stk.pop();
          stk.push(t * v);
        } else {
          int t = stk.top();
          stk.pop();
          stk.push(t / v);
        }
        sign = c;
        v = 0;
      }
    }
    int ans = 0;
    while (!stk.empty()) {
      ans += stk.top();
      stk.pop();
    }
    return ans;
  }
};
```

22 LC 946 | Validate Stack Sequences (Medium)

Problem: Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Constraints: 1 <= pushed.length <= 1000; 0 <= pushed[i] <= 1000; All the elements of pushed are unique.; popped.length == pushed.length; popped is a permutation of pushed.

```
class Solution {
public:
  bool validateStackSequences(vector<int>& pushed,
    vector<int>& popped) {
    stack<int> stk;
    int i = 0;
    for (int x : pushed) {
      stk.push(x);
      while (stk.size() && stk.top() == popped[i])
        {
        stk.pop();
        ++i;
      }
    }
    return i == popped.size();
  }
};
```

15 LC 224 | Basic Calculator (Hard)

Problem: Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation. Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().
Constraints: 1 <= s.length <= 3 * 10^5; s consists of digits, '+', '-', '(', ')', and ' '.; s represents a valid expression.; '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).; '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).; There will be no two consecutive operators in the input.; Every number and running calculation will fit in a signed 32-bit integer.

```
class Solution {
public:
  int calculate(string s) {
    stack<int> stk;
    int ans = 0, sign = 1;
    int n = s.size();
    for (int i = 0; i < n; ++i) {
      if (isdigit(s[i])) {
        int x = 0;
        int j = i;
        while (j < n && isdigit(s[j])) {
          x = x * 10 + (s[j] - '0');
          ++j;
        }
        ans += sign * x;
        i = j - 1;
      } else if (s[i] == '+') {
        sign = 1;
      } else if (s[i] == '-') {
        sign = -1;
      } else if (s[i] == '(') {
        stk.push(ans);
        stk.push(sign);
        ans = 0;
        sign = 1;
      } else if (s[i] == ')') {
        ans *= stk.top();
        stk.pop();
        ans += stk.top();
        stk.pop();
      }
    }
    return ans;
  }
};
```

41 LC 1823 | Find the Winner of the Circular Game (Medium)

Problem: Friends 1..n form a circle. Starting at friend 1, repeatedly count k remaining friends clockwise, including the current one, remove the kth, and resume at the next remaining friend. Return the final winner's label.
Constraints: 1 <= k <= n <= 500

```
class Solution {
public:
  int findTheWinner(int n, int k) {
    int survivor = 0;
    for (int size = 2; size <= n; ++size) survivor
      = (survivor + k) % size;
    return survivor + 1;
  }
};
```

16 LC 32 | Longest Valid Parentheses (Hard)

Problem: Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.
Constraints: 0 <= s.length <= 3 * 10^4; s[i] is '(', or ')'.

```
class Solution {
public:
  int longestValidParentheses(string s) {
    int open = 0, close = 0, best = 0;
    for (char c : s) {
      c == '(' ? ++open : ++close;
      if (open == close) best = max(best, 2 *
        close);
      else if (close > open) open = close = 0;
    }
    open = close = 0;
    for (auto it = s.rbegin(); it != s.rend();
      ++it) {
      *it == '(' ? ++open : ++close;
      if (open == close) best = max(best, 2 *
        open);
      else if (open > close) open = close = 0;
    }
    return best;
  }
};
```

34 LC 239 | Sliding Window Maximum (Hard)

Problem: Slide a window of k consecutive elements across nums, advancing one position at a time. Return the maximum value in each window, in window order.
Constraints: 1 <= nums.length <= 10^5; -10^4 <= nums[i] <= 10^4; 1 <= k <= nums.length

```
class Solution {
public:
  vector<int> maxSlidingWindow(vector<int>& nums,
    int k) {
    deque<int> q;
    vector<int> ans;
    for (int i = 0; i < nums.size(); ++i) {
      if (q.size() && i - q.front() >= k) {
        q.pop_front();
      }
      while (q.size() && nums[q.back()] <=
        nums[i]) {
        q.pop_back();
      }
      q.push_back(i);
      if (i >= k - 1) {
        ans.push_back(nums[q.front()]);
      }
    }
    return ans;
  }
};
```

18 LC 316 | Remove Duplicate Letters (Hard)

Problem: Given a string s, remove duplicate letters so that every letter appears once and only once. You must make sure your result is the smallest in lexicographical order among all possible results.
Constraints: 1 <= s.length <= 10^4; s consists of lowercase English letters.

```
class Solution {
public:
  string removeDuplicateLetters(string s) {
    int n = s.size();
    int last[26] = {0};
    for (int i = 0; i < n; ++i) {
      last[s[i] - 'a'] = i;
    }
    string ans;
    int mask = 0;
    for (int i = 0; i < n; ++i) {
      char c = s[i];
      if ((mask >> (c - 'a')) & 1) {
        continue;
      }
      while (!ans.empty() && ans.back() > c &&
        last[ans.back() - 'a'] > i) {
        mask ^= 1 << (ans.back() - 'a');
        ans.pop_back();
      }
      ans.push_back(c);
      mask |= 1 << (c - 'a');
    }
    return ans;
  }
};
```

39 LC 649 | Dota2 Senate (Medium)

Problem: Radiant ('R') and Dire ('D') senators act cyclically. Each active senator may permanently ban an opposing senator's future turns; a party wins once only its senators remain. Assuming optimal choices, return 'Radiant' or 'Dire'.
Constraints: n == senate.length; 1 <= n <= 10^4; senate[i] is either 'R' or 'D'.

```
class Solution {
public:
  string predictPartyVictory(string senate) {
    int n = senate.size();
    queue<int> qr;
    queue<int> qd;
    for (int i = 0; i < n; ++i) {
      if (senate[i] == 'R') {
        qr.push(i);
      } else {
        qd.push(i);
      }
    }
    while (!qr.empty() && !qd.empty()) {
      int r = qr.front();
      int d = qd.front();
      qr.pop();
      qd.pop();
      if (r < d) {
        qr.push(r + n);
      } else {
        qd.push(d + n);
      }
    }
    return qr.empty() ? "Dire" : "Radiant";
  }
};
```

21 LC 735 | Asteroid Collision (Medium)

Problem: Asteroids move at equal speed along a line: sign gives direction, absolute value size. Oppositely moving asteroids that meet destroy the smaller; equal sizes destroy both. Return survivors in their original order.
Constraints: 2 <= asteroids.length <= 10^4; -1000 <= asteroids[i] <= 1000; asteroids[i] != 0

```
class Solution {
public:
  vector<int> asteroidCollision(vector<int>&
    asteroids) {
    vector<int> stk;
    for (int x : asteroids) {
      if (x > 0) {
        stk.push_back(x);
      } else {
        while (stk.size() && stk.back() > 0 &&
          stk.back() < -x) {
          stk.pop_back();
        }
        if (stk.size() && stk.back() == -x) {
          stk.pop_back();
        } else if (stk.empty() || stk.back() < 0)
          {
          stk.push_back(x);
        }
      }
    }
    return stk;
  }
};
```

27 LC 678 | Valid Parenthesis String (Medium)

Problem: Return whether s can be balanced by interpreting each '*' independently as '(', ')', or an empty string. All fixed parentheses must match in order.
Constraints: 1 <= s.length <= 100; s[i] is '(', ')' or '*'.

```
class Solution {
public:
  bool checkValidString(string s) {
    int x = 0, n = s.size();
    for (int i = 0; i < n; ++i) {
      if (s[i] != ')') {
        ++x;
      } else if (x) {
        --x;
      } else {
        return false;
      }
    }
    x = 0;
    for (int i = n - 1; i >= 0; --i) {
      if (s[i] != '(') {
        ++x;
      } else if (x) {
        --x;
      } else {
        return false;
      }
    }
    return true;
  }
};
```

23 LC 1047 | Remove All Adjacent Duplicates In String (Easy)

Problem: You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters and removing them. We repeatedly make duplicate removals on s until we no longer can. Return the final string after all such duplicate removals have been made. It can be proven that the answer is unique.

Constraints: 1 <= s.length <= 10^5; s consists of lowercase English letters.

```
class Solution {
public:
  string removeDuplicates(string s) {
    string stk;
    for (char c : s) {
      if (!stk.empty() && stk[stk.size() - 1] ==
        c) {
        stk.pop_back();
      } else {
        stk += c;
      }
    }
    return stk;
  }
};
```

32 LC 622 | Design Circular Queue (Medium)

Problem: Implement MyCircularQueue(k): enQueue(value)/deQueue() report success; insertions fail when full and deletions when empty. Front()/Rear() return end values or -1 when empty. isEmpty()/isFull() report state. Use circular storage; do not use a built-in queue.

Constraints: 1 <= k <= 1000; 0 <= value <= 1000; At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

```
class MyCircularQueue {
private:
  int front;
  int size;
  int capacity;
  vector<int> q;
public:
  MyCircularQueue(int k) {
    capacity = k;
    q = vector<int>(k);
    front = size = 0;
  }
  bool enQueue(int value) {
    if (isFull()) return false;
    int idx = (front + size) % capacity;
    q[idx] = value;
    ++size;
    return true;
  }
  bool deQueue() {
    if (isEmpty()) return false;
    front = (front + 1) % capacity;
    --size;
    return true;
  }
  int Front() {
    if (isEmpty()) return -1;
    return q[front];
  }
  int Rear() {
    if (isEmpty()) return -1;
    int idx = (front + size - 1) % capacity;
    return q[idx];
  }
  bool isEmpty() {
    return size == 0;
  }
  bool isFull() {
    return size == capacity;
  }
};
```

24 LC 1209 | Remove All Adjacent Duplicates in String II (Medium)

Problem: You are given a string s and an integer k, a k duplicate removal consists of choosing k adjacent and equal letters from s and removing them, causing the left and the right side of the deleted substring to concatenate together. We repeatedly make k duplicate removals on s until we no longer can. Return the final string after all such duplicate removals have been made. It is guaranteed that the answer is unique.

Constraints: 1 <= s.length <= 10^5; 2 <= k <= 10^4; s only contains lowercase English letters.

```
class Solution {
public:
  string removeDuplicates(string s, int k) {
    vector<pair<char, int>> stk;
    for (char& c : s) {
      if (stk.size() && stk.back().first == c) {
        stk.back().second = (stk.back().second +
          1) % k;
        if (stk.back().second == 0) {
          stk.pop_back();
        }
      } else {
        stk.push_back({c, 1});
      }
    }
    string ans;
    for (auto [c, v] : stk) {
      ans += string(v, c);
    }
    return ans;
  }
};
```

49 LC 210 | Course Schedule II (Medium)

Problem: Return an order for all numCourses courses such that every prerequisite appears before its dependent course. Pair [a,b] means b must precede a. Return an empty array if impossible; any valid order is accepted.

Constraints: 1 <= numCourses <= 2000; 0 <= prerequisites.length <= numCourses * (numCourses - 1); prerequisites[i].length == 2; 0 <= a_i, b_i < numCourses; a_i != b_i; All the pairs [a_i, b_i] are distinct.

```
class Solution {
public:
  vector<int> findOrder(int numCourses,
    vector<vector<int>>& prerequisites) {
    vector<vector<int>> g(numCourses);
    vector<int> indeg(numCourses);
    for (auto& p : prerequisites) {
      int a = p[0], b = p[1];
      g[b].push_back(a);
      ++indeg[a];
    }
    queue<int> q;
    for (int i = 0; i < numCourses; ++i) {
      if (indeg[i] == 0) {
        q.push(i);
      }
    }
    vector<int> ans;
    while (!q.empty()) {
      int i = q.front();
      q.pop();
      ans.push_back(i);
      for (int j : g[i]) {
        if (--indeg[j] == 0) {
          q.push(j);
        }
      }
    }
    return ans.size() == numCourses ? ans :
      vector<int>();
  }
};
```

26 LC 1544 | Make The String Great (Easy)
Problem: Repeatedly remove adjacent occurrences of the same English letter in opposite case until no such pair remains. Return the final string, which is unique.
Constraints: 1 <= s.length <= 100; s contains only lower and upper case English letters.

```
class Solution {
public:
  string makeGood(string s) {
    string stk;
    for (char c : s) {
      if (stk.empty() || abs(stk.back() - c) !=
        32) {
        stk += c;
      } else {
        stk.pop_back();
      }
    }
    return stk;
  }
};
```

30 LC 1021 | Remove Outermost Parentheses (Easy)
Problem: Split a valid parentheses string into its primitive balanced components. Remove the outermost matching pair from every component and concatenate the remaining strings.
Constraints: 1 <= s.length <= 10^5; s[i] is either '(' or ')'.; s is a valid parentheses string.

```
class Solution {
public:
  string removeOuterParentheses(string s) {
    string ans;
    int cnt = 0;
    for (char& c : s) {
      if (c == '(') {
        if (++cnt > 1) {
          ans.push_back(c);
        }
      } else {
        if (--cnt) {
          ans.push_back(c);
        }
      }
    }
    return ans;
  }
};
```

37 LC 1696 | Jump Game VI (Medium)
Problem: Start at nums[0]. Each jump advances 1..k positions, without passing the final index. Score is the sum of visited values, including both ends. Return the maximum score reaching the last index.
Constraints: 1 <= nums.length, k <= 10^5; -10^4 <= nums[i] <= 10^4

```
class Solution {
public:
  int maxResult(vector<int>& nums, int k) {
    deque<pair<int, int>> candidates;
    int score = 0;
    for (int i = 0; i <
      static_cast<int>(nums.size()); ++i) {
      while (!candidates.empty() &&
        candidates.front().first < i - k)
        candidates.pop_front();
      score = nums[i] + (candidates.empty() ? 0 :
        candidates.front().second);
      while (!candidates.empty() &&
        candidates.back().second <= score)
        candidates.pop_back();
      candidates.emplace_back(i, score);
    }
    return score;
  }
};
```

28 LC 921 | Minimum Add to Make Parentheses Valid (Medium)
Problem: Return the minimum number of '(' or ')' insertions needed to make s balanced. Insertions may be made at any positions.
Constraints: 1 <= s.length <= 1000; s[i] is either '(' or ')'.

```
class Solution {
public:
  int minAddToMakeValid(string s) {
    int ans = 0, cnt = 0;
    for (char c : s) {
      if (c == '(')
        ++cnt;
      else if (cnt)
        --cnt;
      else
        ++ans;
    }
    ans += cnt;
    return ans;
  }
};
```

40 LC 1700 | Number of Students Unable to Eat Lunch (Easy)
Problem: Students queue for square (1) or circular (0) sandwiches. The front student takes the top sandwich if it matches; otherwise moves to the back. Stop when nobody remaining wants the top sandwich. Return the number unable to eat.
Constraints: 1 <= students.length, sandwiches.length <= 100; students.length == sandwiches.length; sandwiches[i] is 0 or 1.; students[i] is 0 or 1.

```
class Solution {
public:
  int countStudents(vector<int>& students,
    vector<int>& sandwiches) {
    int cnt[2] = {0};
    for (int& v : students) ++cnt[v];
    for (int& v : sandwiches) {
      if (cnt[v]-- == 0) {
        return cnt[v ^ 1];
      }
    }
    return 0;
  }
};
```

42 LC 950 | Reveal Cards In Increasing Order (Medium)
Problem: Arrange distinct cards so repeatedly revealing the top card and moving the next top card to the bottom reveals values in increasing order. Return the initial top-to-bottom arrangement.
Constraints: 1 <= deck.length <= 1000; 1 <= deck[i] <= 10^6; All the values of deck are unique.

```
class Solution {
public:
  vector<int> deckRevealedIncreasing(vector<int>&
    deck) {
    sort(deck.rbegin(), deck.rend());
    deque<int> q;
    for (int v : deck) {
      if (!q.empty()) {
        q.push_front(q.back());
        q.pop_back();
      }
      q.push_front(v);
    }
    return vector<int>(q.begin(), q.end());
  }
};
```

29 LC 856 | Score of Parentheses (Medium)

Problem: Given a balanced parentheses string s, return the score of the string. The score of a balanced parentheses string is based on the following rule: - "()" has score 1. - AB has score A + B, where A and B are balanced parentheses strings. - (A) has score 2 * A, where A is a balanced parentheses string.

Constraints: 2 <= s.length <= 50; s consists of only '(' and ')'.; s is a balanced parentheses string.

```
class Solution {
public:
  int scoreOfParentheses(string s) {
    int ans = 0, d = 0;
    for (int i = 0; i < s.size(); ++i) {
      if (s[i] == '(') {
        ++d;
      } else {
        --d;
        if (s[i - 1] == '(') {
          ans += 1 << d;
        }
      }
    }
    return ans;
  }
};
```

47 LC 752 | Open the Lock (Medium)

Problem: A four-wheel lock starts at '0000'. Each move turns one wheel one step forward or backward with digit wraparound. Avoid every deadend, including the starting state. Return minimum moves to target, or -1 if impossible.

Constraints: 1 <= deadends.length <= 500; deadends[i].length == 4; target.length == 4; target will not be in the list deadends.; target and deadends[i] consist of digits only.

```
class Solution {
public:
  int openLock(vector<string>& deadends, string
    target) {
    unordered_set<string> s(deadends.begin(),
      deadends.end());
    if (s.count("0000")) return -1;
    if (target == "0000") return 0;
    queue<string> q{{"0000"}};
    s.insert("0000");
    int ans = 0;
    while (!q.empty()) {
      ++ans;
      for (int n = q.size(); n > 0; --n) {
        string p = q.front();
        q.pop();
        for (string t : next(p)) {
          if (target == t) return ans;
          if (!s.count(t)) {
            q.push(t);
            s.insert(t);
          }
        }
      }
    }
    return -1;
  }
  vector<string> next(string& t) {
    vector<string> res;
    for (int i = 0; i < 4; ++i) {
      char c = t[i];
      t[i] = c == '0' ? '9' : (char) (c - 1);
      res.push_back(t);
      t[i] = c == '9' ? '0' : (char) (c + 1);
      res.push_back(t);
      t[i] = c;
    }
    return res;
  }
};
```

33 LC 641 | Design Circular Deque (Medium)

Problem: Implement MyCircularDeque(k), a circular deque of capacity k. insertFront(value), insertLast(value), deleteFront(), and deleteLast() return success; insertions fail when full and deletions when empty. getFront()/getRear() return the end values or -1 when empty. isEmpty()/isFull() report status.

Constraints: 1 <= k <= 1000; 0 <= value <= 1000; At most 2000 calls will be made to insertFront, insertLast, deleteFront, deleteLast, getFront, getRear, isEmpty, isFull.

```
class MyCircularDeque {
public:
  vector<int> q;
  int front = 0;
  int size = 0;
  int capacity = 0;
  MyCircularDeque(int k) {
    q.assign(k, 0);
    capacity = k;
  }
  bool insertFront(int value) {
    if (isFull()) {
      return false;
    }
    if (!isEmpty()) {
      front = (front - 1 + capacity) % capacity;
    }
    q[front] = value;
    ++size;
    return true;
  }
  bool insertLast(int value) {
    if (isFull()) {
      return false;
    }
    int idx = (front + size) % capacity;
    q[idx] = value;
    ++size;
    return true;
  }
  bool deleteFront() {
    if (isEmpty()) {
      return false;
    }
    front = (front + 1) % capacity;
    --size;
    return true;
  }
  bool deleteLast() {
    if (isEmpty()) {
      return false;
    }
    --size;
    return true;
  }
  int getFront() {
    return isEmpty() ? -1 : q[front];
  }
  int getRear() {
    return isEmpty() ? -1 : q[(front + size - 1) %
      capacity];
  }
  bool isEmpty() {
    return size == 0;
  }
  bool isFull() {
    return size == capacity;
  }
};
```

36 LC 862 | Shortest Subarray with Sum at Least K (Hard)

Problem: Given an integer array nums and an integer k, return the length of the shortest non-empty subarray of nums with a sum of at least k. If there is no such subarray, return -1. A subarray is a contiguous part of an array.

Constraints: 1 <= nums.length <= 10^5; -10^5 <= nums[i] <= 10^5; 1 <= k <= 10^9

```
class Solution {
public:
  int shortestSubarray(vector<int>& nums, int k) {
    int n = nums.size();
    vector<long> s(n + 1);
    for (int i = 0; i < n; ++i) s[i + 1] = s[i] +
      nums[i];
    deque<int> q;
    int ans = n + 1;
    for (int i = 0; i <= n; ++i) {
      while (!q.empty() && s[i] - s[q.front()] >=
        k) {
        ans = min(ans, i - q.front());
        q.pop_front();
      }
      while (!q.empty() && s[q.back()] >= s[i])
        q.pop_back();
      q.push_back(i);
    }
    return ans > n ? -1 : ans;
  }
};
```

44 LC 102 | Binary Tree Level Order Traversal (Medium)

Problem: Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Constraints: The number of nodes in the tree is in the range [0, 2000].; -1000 <= Node.val <= 1000

```
class Solution {
public:
  vector<vector<int>> levelOrder(TreeNode* root) {
    vector<vector<int>> ans;
    if (!root) return ans;
    queue<TreeNode*> q{{root}};
    while (!q.empty()) {
      vector<int> t;
      for (int n = q.size(); n; --n) {
        auto node = q.front();
        q.pop();
        t.push_back(node->val);
        if (node->left) {
          q.push(node->left);
        }
        if (node->right) {
          q.push(node->right);
        }
      }
      ans.push_back(t);
    }
    return ans;
  }
};
```

48 LC 127 | Word Ladder (Hard)

Problem: Return the number of words in a shortest transformation from beginWord to endWord, changing one letter per step. Every subsequent word must be in wordList; beginWord need not. Return 0 if impossible.

Constraints: 1 <= beginWord.length <= 10; endWord.length == beginWord.length; 1 <= wordList.length <= 5000; wordList[i].length == beginWord.length; beginWord, endWord, and wordList[i] consist of lowercase English letters.; beginWord != endWord; All the words in wordList are unique.

```
class Solution {
public:
  int ladderLength(string beginWord, string
    endWord, vector<string>& wordList) {
    unordered_set<string> words(wordList.begin(),
      wordList.end());
    if (!words.count(endWord)) return 0;
    queue<string> q1{{beginWord}};
    queue<string> q2{{endWord}};
    unordered_map<string, int> m1;
    unordered_map<string, int> m2;
    m1[beginWord] = 0;
    m2[endWord] = 0;
    while (!q1.empty() && !q2.empty()) {
      int t = q1.size() <= q2.size() ? extend(m1,
        m2, q1, words) : extend(m2, m1, q2,
        words);
      if (t != -1) return t + 1;
    }
    return 0;
  }
  int extend(unordered_map<string, int>& m1,
    unordered_map<string, int>& m2, queue<string>&
    q, unordered_set<string>& words) {
    for (int i = q.size(); i > 0; --i) {
      string s = q.front();
      int step = m1[s];
      q.pop();
      for (int j = 0; j < s.size(); ++j) {
        char ch = s[j];
        for (char k = 'a'; k <= 'z'; ++k) {
          s[j] = k;
          if (!words.count(s) || m1.count(s))
            continue;
          if (m2.count(s)) return step + 1 +
            m2[s];
          m1[s] = step + 1;
          q.push(s);
        }
        s[j] = ch;
      }
    }
    return -1;
  }
};
```